CS 146, Data Structures and Algorithms, SJSU: Doug Case
Program 2:
Bubble Sort vs. Short Circuiting Sort vs. (Lomuto partitioning) QuickSort

1) Copy the given code below into an IDE:

```
package package1;  // NOTE: Students might not need this, or might need to change this
import java.util.Random;

// Java program for implementations of QuickSort
public class QuickSort
{
 /* This method takes last element as pivot, places the pivot element
   at its correct position in sorted array, and places all smaller
   (smaller than pivot) to left of pivot and all greater elements
   to the right of pivot.
   This is most-frequently seen implementation of QuickSort partition,
   and this is also known as the Lomuto partition */
   int partition(int arr[], int low, int high)
   {
      int pivot = arr[high];
      int i = (low - 1); // index of smaller element
      for (int j = low; j < high; j++)
      {
         // If current element is smaller than or
         // equal to pivot
         if (arr[j] <= pivot)
         {
            i++;

            // swap arr[i] and arr[j]
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
         }
      }

      // swap arr[i+1] and arr[high] (or pivot)
      int temp = arr[i + 1];
      arr[i + 1] = arr[high];
      arr[high] = temp;

      return i + 1;
   }

   /* The main method that implements QuickSort()
     arr[] --> Array to be sorted,
     low  --> Starting index,
     high  --> Ending index.
     This sorts the array, and returns the time (in ms) it takes to run */
   long sort(int arr[], int low, int high)
```

```
{
        long startTime = System.currentTimeMillis();
   if (low < high)
   {
      /* partIdx is partitioning index, arr[partIdx] is now at right place */
      int partIdx = partition(arr, low, high);

      // Recursively sort elements before partition and after partition
      sort(arr, low, partIdx - 1);
      sort(arr, partIdx + 1, high);
   }
   return System.currentTimeMillis() - startTime;
}

/* The main method that implements QuickSort() with Hoare partitioning
  TODO: Implement this method and make a new method called partitionHoare
       that is conceptually similar to the partition method in this class.
       But instead of using the "default" (Lomuto) partitioning use
       the Hoare partitioning, as described in Prof. David Taylor's
       https://www.youtube.com/watch?v=v-1EGgaTFuw
       and elsewhere.
   This sorts the array, and returns the time (in ms) it takes to run */
long sortHoare(int arr[], int low, int high)
{
        return 0; // temporary, replace this
   // TODO...
}

/* This is a method to print array of size n.
 * NOTE: use of this method is optional.
 * But this can be helpful to debug your code to make sure it really sorts.
 * If you reduce the main's NUM_ITEMS to some manageable number
 * (like 20 or whatever) you can debug your code, and then
 * reset main's NUM_ITEMS to a large value to make your measurements */
static void printArray(int arr[])
{
   int n = arr.length;
   for (int i = 0; i < n; ++i)
   {
      System.out.print(arr[i] + " ");
   }
   System.out.println();
}

/* method to get an already sorted array of size arraySize */
static int[] getAscendingArray(int arraySize)
{
   int[] newArray = new int[arraySize];
   for (int i = 0; i < arraySize; i++)
   {
      newArray[i] = i * 3; // arbitrarily increment by 3 each time
   }
   return newArray;
}

/* method to get an array of size arraySize in descending order */
static int[] getDescendingArray(int arraySize)
{
   int[] newArray = new int[arraySize];
   int n = 0;
   for (int i = arraySize - 1; i >= 0; i--)
   {
```

```
        n += 4;          // arbitrarily increment from end by 4 each time
        newArray[i] = n ; // gets the array in descending order
    }
    return newArray;
}


/* method to get an array of size arraySize in descending order */
static int[] getRandomArray(int arraySize)
{
    Random rand = new Random();
    // Setting upper bound to generate random numbers in specific range
    int upperbound = 1000000;
    int[] newArray = new int[arraySize];
    for (int i = 0; i < arraySize; i++)
    {
        // Generating random values from 0 - 999999 using nextInt()
        newArray[i] = rand.nextInt(upperbound);
    }
    return newArray;
}

/* method to get an array of size arraySize with all values the same */
static int[] getSameArray(int arraySize)
{
            int[] newArray = new int[arraySize];
    for (int i = 0; i < arraySize; i++)
    {
        newArray[i] = 42; // arbitrary value, same for all elements
    }
    return newArray;
}

/* This is a bubble sort, runtime n ** 2 worst case.
 * TODO: If trySHortCircuit parameter is true, then
 * when and if the array is already sorted then
 * stop, no need to keep sorting, just return. */
long bubbleSort(int arr[], boolean trySHortCircuit)
{
            // TODO - actual do something with parameter trySHortCircuit
            long startTime = System.currentTimeMillis();
    int len = arr.length;
    for (int i = 0; i < len - 1; i++)
    {
        for (int j = 0; j < len - 1; j++)
        {
            if (arr[j] > arr[j+1]) {
             int temp = arr[j+1];
             arr[j+1] = arr[j];
             arr[j] = temp;
            }
        }
    }
    return System.currentTimeMillis() - startTime;
}

// main driver program
public static void main(String args[])
{
            // NOTE: if you get StackOverflowError then decrease NUM_ITEMS below
    final int NUM_ITEMS = 6000;
    QuickSort ob = new QuickSort();
```

```
int arr[] = getAscendingArray(NUM_ITEMS);

long ascTime = ob.sort(arr, 0, NUM_ITEMS - 1);

arr = getDescendingArray(NUM_ITEMS);
long descTime = ob.sort(arr, 0, NUM_ITEMS - 1);

arr = getRandomArray(NUM_ITEMS);
long randomTime = ob.sort(arr, 0, NUM_ITEMS - 1);
//printArray(arr);  // NOTE: can verify sorted correctly (for small arrays)

arr = getSameArray(NUM_ITEMS);
long sameTime = ob.sort(arr, 0, NUM_ITEMS - 1);

System.out.println("QuickSort times using Lomuto (default) partitioning follow:");
System.out.println("sorted asc array, time in ms:    " + Long.toString(ascTime));
System.out.println("sorted desc array, time in ms:   " + Long.toString(descTime));
System.out.println("sorted random array, time in ms: " + Long.toString(randomTime));
System.out.println("sorted same values array, time:  " + Long.toString(sameTime));

arr = getAscendingArray(NUM_ITEMS);
ascTime = ob.sortHoare(arr, 0, NUM_ITEMS - 1);

arr = getDescendingArray(NUM_ITEMS);
descTime = ob.sortHoare(arr, 0, NUM_ITEMS - 1);

arr = getRandomArray(NUM_ITEMS);
randomTime = ob.sortHoare(arr, 0, NUM_ITEMS - 1);
//printArray(arr);  // NOTE: can verify sorted correctly (for small arrays)

arr = getSameArray(NUM_ITEMS);
sameTime = ob.sortHoare(arr, 0, NUM_ITEMS - 1);

System.out.println("\nQuickSort times using Hoare partitioning follow:");
System.out.println("sorted asc array, time in ms:    " + Long.toString(ascTime));
System.out.println("sorted desc array, time in ms:   " + Long.toString(descTime));
System.out.println("sorted random array, time in ms: " + Long.toString(randomTime));
System.out.println("sorted same values array, time:  " + Long.toString(sameTime));

arr = getAscendingArray(NUM_ITEMS);
ascTime = ob.bubbleSort(arr, false);

arr = getDescendingArray(NUM_ITEMS);
descTime = ob.bubbleSort(arr, false);

arr = getRandomArray(NUM_ITEMS);
randomTime = ob.bubbleSort(arr, false);

arr = getSameArray(NUM_ITEMS);
sameTime = ob.bubbleSort(arr, false);
System.out.println("\nBubble Sort times follow:");
System.out.println("sorted asc array, time in ms:    " + Long.toString(ascTime));
System.out.println("sorted desc array, time in ms:   " + Long.toString(descTime));
System.out.println("sorted random array, time in ms: " + Long.toString(randomTime));
System.out.println("sorted same values array, time:  " + Long.toString(sameTime));

arr = getAscendingArray(NUM_ITEMS);
ascTime = ob.bubbleSort(arr, true);

arr = getDescendingArray(NUM_ITEMS);
descTime = ob.bubbleSort(arr, true);
```

```
    arr = getRandomArray(NUM_ITEMS);
    randomTime = ob.bubbleSort(arr, true);
    //printArray(arr);  // NOTE: can verify sorted correctly (for small arrays)

    arr = getSameArray(NUM_ITEMS);
    sameTime = ob.bubbleSort(arr, true);
    System.out.println("\nBubble Sort times with possible short circuit follow:");
    System.out.println("sorted asc array, time in ms:    " + Long.toString(ascTime));
    System.out.println("sorted desc array, time in ms:   " + Long.toString(descTime));
    System.out.println("sorted random array, time in ms: " + Long.toString(randomTime));
    System.out.println("sorted same values array, time:  " + Long.toString(sameTime));
  }
}
```

2) For this assignment you will change code in main, change the NUM_ITEMS constant. You might optionally want to make it very small while debugging, then change it back to larger when it is working.

```
  /* This is a bubble sort, runtime n ** 2 worst case.
   * TODO: If tryShortCircuit parameter is true, then
   * when and if the array is already sorted then
   * stop, no need to keep sorting, just return. */
  long bubbleSort(int arr[], boolean tryShortCircuit)
  {
    // TODO - actual do something with parameter tryShortCircuit
    long startTime = System.currentTimeMillis();
    int len = arr.length;
    for (int i = 0; i < len - 1; i++)
    {
      for (int j = 0; j < len - 1; j++)
      {
        if (arr[j] > arr[j+1]) {
         int temp = arr[j+1];
         arr[j+1] = arr[j];
         arr[j] = temp;
        }
      }
    }
    return System.currentTimeMillis() - startTime;
  }
```

3) Add code in bubbleSort() method above to detect if array is sorted before doing the whole n-squared operations, and short-circuit in that case – that is, stop sorting and return early. This shouldn't be much code.
4) Add code in sortHoare() to actually sort, and add a method called partitionHoare() and call it.
5) Run the code, contrast the performance in the 4 data scenarios with Quicksort using the default Lomuto quicksort that is included, and contrast it with Quicksort using sortHoare() and partitionHoare(). Submit the output, and a paragraph or so with your explanation of the differences.
6) Experiment with increasing NUM_ITEMS and see at around what number the quicksort fails on your computer. Then, comment out the calls to QuickSort, and experiment with increasing NUM_ITEMS and try running the bubble sort and see what happens. Include your code in the output. Include your output in the writeup. Write a couple paragraphs with your guess/estimation/theory to explain the results you

see. All reasonable guesses will receive reasonably generous grading.